

AN EFFICIENT TASK SCHEDULING ALGORITHM IN CLOUD ENVIRONMENT

A project work submitted to the Jamal Mohamed College (Autonomous), Tiruchirappalli
in partial fulfillment of the requirements for the award of the degree of

MASTER OF SCIENCE IN COMPUTER SCIENCE

by

HANSIKA J
(Register Number: 23PCS062)

Guided by

Mrs. S.PRABHAVATHI, M.Sc., M.Phil., B.Ed.,
ASSISTANT PROFESSOR



DEPARTMENT OF COMPUTER SCIENCE & IT
JAMAL MOHAMED COLLEGE (AUTONOMOUS)

Accredited with 'A++' Grade by NAAC (4th Cycle) with CGPA 3.69 out of 4.0

(Affiliated to Bharathidasan University)

TIRUCHIRAPPALLI-620 020

APRIL 2025

JAMAL MOHAMED COLLEGE (AUTONOMOUS)

Accredited with 'A++' Grade by NAAC (4th Cycle) with CGPA 3.69 out of 4.0

(Affiliated to Bharathidasan University)

TIRUCHIRAPPALLI-620 020



This is to certify that this project work entitled

**AN EFFICIENT TASK SCHEDULING ALGORITHM IN CLOUD
ENVIRONMENT**

is a Bonafide record of the project work done

by

HANSIKA J

23PCS062

at Jamal Mohamed College (Autonomous), Tiruchirappalli during the year 2024-2025

in partial fulfillment of the requirements for the award of the degree of

MASTER OF SCIENCE IN COMPUTER SCIENCE

Guide

Head of the Department

Submitted for Viva-Voce examination held on _____ at Jamal

Mohamed College, Tiruchirappalli-620 020.

Internal Examiner

External Examiner

Mrs. S.PRABHAVATHI, M.Sc., M.Phil., B.Ed.,
Assistant Professor,
Department of Computer Science,
Jamal Mohamed College (Autonomous),
Tiruchirappalli – 620 020.

DATE: _____

CERTIFICATE

This is to certify that this project entitled **“AN EFFICIENT TASK SCHEDULING ALGORITHM IN CLOUD ENVIRONMENT”** submitted in partial fulfillment of the requirements for the award of the degree of Master of Science in Computer Science to the Jamal Mohamed College, Tiruchirappalli is a Bonafide record of the work done by **Hansika J (Register Number: 23PCS062)** under my supervision and guidance.

Place: Tiruchirappalli-20

Date:

Signature of the Guide

DECLARATION

I hereby declare that dissertation entitled "**AN EFFICIENT TASK SCHEDULING ALGORITHM IN CLOUD ENVIRONMENT**" submitted to Jamal Mohamed College (Autonomous), Tiruchirappalli, in partial fulfillment of the requirements for the degree of Master of Science in Computer Science (Full Time) is a record of original research work done by **HANSIKA J (23PCS062)** during the period of the study in the Department of Computer Science degree, Jamal Mohamed College (Autonomous), Tiruchirappalli, under the guidance of **Mrs. S.PRABHAVATHI**, Assistant Professor, in Department of Computer Science in Jamal Mohamed College and that the dissertation has not previously formed the basis for the award of any Degree / Diploma or other similar title.

Place: Tiruchirappalli-20

Date:

Signature of the Candidate

ACKNOWLEDGEMENT

Gratitude is a sense of thankfulness to the hands that stretched out towards us in help. At the onset, I am grateful to the Lord Almighty who has showered his grace and enlightened me through this project. A journey of miles begins with a single step. Words are inadequate to express my heartfelt, sincere, and grateful thanks to the following persons.

My first teachers and well-wishers, our first guide, I first express our deepest gratitude to our parents who were our strength and for their constant support in our every single step.

I extend my heartfelt thanks to the management for providing an excellent platform and resources that facilitated this work.

I am deeply grateful to our Principal, **Dr. D.I. GEORGE AMALARETHINAM**, for his constant encouragement and invaluable support in facilitating this project work.

A special note of gratitude goes to our **Dr. M. MOHAMED SURPUTHEEN**, Head of the Department, Department of Computer Science and IT, for their continuous support, advice, and encouragement in successfully executing this project.

I am immensely thankful to our **DR. S.VAAHEEDHA KFATHEEN**, Dean of Science, Department of Computer Science, whose expertise, patience, and constructive feedback have been instrumental in the successful completion of this work. Her continuous support and encouragement have been invaluable.

I also extend my heartfelt appreciation to our Member In-charge of the Department **Mrs. J. FATHIMA FOUZIA**, Associate Professor of Computer Science for their valuable guidance and motivation.

As the pole star guides every lonely traveller with assurance and safety, so has my guide, I extend my sincere thanks to my project Guide, **Mrs. S. PRABHAVATHI, Assistant Professor** of Computer Science for all her guidance, encouragement, and inspiration that enabled me to shine in every walk of life.

Additionally, I extend my appreciation to all the faculty members and staff of the Department of Computer Science, for their assistance, encouragement, and valuable inputs.

Finally, I express my sincere gratitude to my friends and family for their unwavering support and motivation throughout this journey.

HANSIKA J

ABSTRACT

Cloud computing is a recently emerging paradigm. The ultimate aim of cloud computing is to provide on-demand, pay-as-you-go, internet-based access to shared computing resources (hardware and software) in a metered, self-service, dynamically scalable fashion.

Cloud computing is a model for allowing suitable, on demand network access to a shared pool of configurable computing resources (e.g. networks, storage, applications, servers, and services) that can be quickly provisioned and unconfined with minimal management power or service provider interaction.

In order to maximize the effectiveness of cloud computing, we require an efficient scheduling algorithm. Task scheduling is one of the main issues in cloud computing environment. Efficient task scheduling is significant to achieve minimum makespan, cost-effective execution and increase resource utilization.

The objectives of this task scheduling are to identify the most appropriate resources for scheduling a specific task on time and utilize the resources in the most effective way.

In this paper, an effective algorithm is proposed that reduces the total execution time (makespan) and also increases resource utilization. The results of the simulation indicate that the new algorithm designed for heterogeneous systems surpasses several established algorithms, including min-min algorithm, both in terms of makespan and resource utilization.

LIST OF FIGURES

Figure 1 Work flow of the Proposed Algorithm	24
Figure 2 Chart of Min-Min Algorithm.....	31
Figure 3 Chart of Proposed Algorithm.....	31

LIST OF TABLES

Table 1 Sample ETC matrix.....	26
Table 2 Minimum Execution time of all tasks	27
Table 3 ETC matrix1.....	28
Table 4 ETC matrix2.....	28
Table 5 Scheduling using the proposed algorithm	29
Table 6 Scheduling using the Min-Min algorithm	30

ABBRIEVATIONS

NIST	-	National Institute of Standards and Technology
VM	-	Virtual Machine
QoS	-	Quality of Service
SaaS	-	Software as a Service
IaaS	-	Information as a Service
PaaS	-	Platform as a Service
CPU	-	Central Processing Unit
AHP	-	Analytic Hierarchy Process
MCT	-	Minimum Completion Time
GA	-	Genetic Algorithm
ANOVA	-	Analysis of Variance
PPIA	-	Priority Based Performance Improved Algorithm
LBMM	-	Load Balancing Min-Min
REMM	-	Rescheduling Enhanced Min-Min
RAMM	-	Resource Aware Min-Min
DAG	-	Directed Acyclic Graph
PTCT	-	Prediction of Tasks Computation Time Algorithm
PCA	-	Principle Components Analysis
ETC	-	Expected Time to Compute

TABLE OF CONTENTS

ABSTRACT	i
LIST OF FIGURES	ii
LIST OF TABLES	iii
ABBREVIATIONS	iv
1. INTRODUCTION.....	1
2. LITERATURE SURVEY.....	11
3. ANALYSIS AND DRAWBACKS IN EXISTING SYSTEM.....	17
4. STATEMENT OF THE PROBLEM	18
5. SCOPE OF THE RESEARCH	19
6. OBJECTIVES OF THE RESEARCH	20
7. PROPOSED METHODOLOGY	21
8. EXPERIMENTAL RESULT	26
9. CONCLUSION AND FUTURE WORK	32
REFERENCES	33
APPENDIX	35
A) SOURCE CODE.....	35
B) SCREEN INTERFACES	40

CHAPTER 1

INTRODUCTION

CLOUD COMPUTING

Cloud computing revolutionizes the way we access and manage computing resources. Instead of relying on local servers, it leverages the internet to provide on-demand access to a wide range of services, from storage and processing power to applications. This model offers flexibility, scalability, and cost-effectiveness, enabling businesses and individuals to optimize resource utilization and focus on innovation rather than infrastructure management.

According to NIST, it can be thought of as a model for enabling a ubiquitous, convenient and on-demand way of accessing a pool of resources. It has led to the growth of virtual and physical devices, which in turn has led to an escalation of data generated on a continuous basis.

Virtualization techniques act as the backbone of cloud technology. They involve constant allocation and reallocation of resources, which is done at the service provider level and provides the clients with a view of an infinite computing and storage capacity. The service providers are constantly flooded with demands for high-performance computing and newer resources. This constant and varying demand for resources by the clients is what leads to the development of efficient scheduling techniques across a distributed system, such as cloud.

Until recently meeting the demands of the consumers has been the sole concern of service providers, without paying any attention to the optimum allocation of resources. However, even though the number of cloud providers and consumers is increasing, there is also an escalating amount of resource wastage. Moreover, there is a need to maintain high service-level performance along with optimization of the resource usage.

Cloud computing is one of the forms of utility computing in which the user need not own any infrastructure or software or platform needed for them. Instead the user can make use of the service provided by the cloud and they can pay as peruse. Cloud provides dynamic services using the concept of virtualization and scalability.

Cloud can be defined as an execution environment in elastic fashion which involves a metered service for multiple varied users. Some of the services provided by cloud environment are Software as a service (SaaS), Platform as a service (PaaS) and Infrastructure as a service(IaaS).

Cloud technology is the pioneer in implementing commercial form of using the computer science by public users. The concept of virtualization is used to share the available resources among the users who requested the service. The cloud provides a high performance by dividing the workloads among all the available resources efficiently which reduces the execution time, waiting time and throughput. The distributed computing environments provide various services out of which job scheduling is a major activity. The performance can be increased by efficient task scheduling which can also balance the load among the resources.

There are several issues and challenges in cloud computing; cost, security, accessibility, completion time of user tasks, availability, flexibility, performance monitoring, requires a constant & speedy Internet connection, consistent and robust service abstractions, VM allocation and migration, interoperability and portability, task scheduling, scale and QoS management, and efficient load balancing.

Most of researchers consider task scheduling, resource allocation and load balancing are the major concerns in the cloud computing and distribution systems, they play a significance role to enhance overall system performance.

With the advancement and increasing need of information technology, the cloud computing is coming to stay as a suitable solution for both individuals and business needs, provides to them with a large scalable and virtualized cloud resources such as hardware, servers, memory, network, storage, and services in a dynamic manner, on-demand, remote-access and pay-per-use over the world via the internet.

Besides that, the cloud computing has several benefits and characteristics compared to other computing technologies; it is a virtualized, cost- effective, resource pooling, independent of device and location, elastic, broad network access that available all times and accessing from any where via internet or private channels. It saves high expenses for creating data centers, maintenance, disaster recovery, power consumption, and technical staff.

Thus, the main goal of cloud computing is to get best utilization of computing resources and distributed systems, by combining them to achieve higher throughput and to be suitable for large scale systems and organizations.

DEPLOYMENT MODELS

There are four basic types of deployment models in cloud computing these are

- a) **Public cloud:** The public clouds are owned, created, operated and maintained by government organizations or companies that offer the others such as an individual, different companies and various organization to obtain fast access and computation through their cloud infrastructure. This cloud is open to all and accessed by everyone and also it consists of an enormous collection of resources of several types. The users need not have to purchase resources individually rather can access resources from the cloud which is operated and maintained by service provider.
- b) **Private cloud:** The private cloud is a cloud that is created and owned by an individual or a single organization. The resources contained in this cloud specifically belong to the owner and is accessed by the ones who have permission to access those resources. The cloud infrastructure build is exclusively formulated for a single organization which is maintained and managed internally or by a third-party.
- c) **Community cloud:** The cloud which is created and shared among a specific community is known as community cloud. The resources that it encompasses belong to specific community users and hence the interactions are limited to that community user only.
- d) **Hybrid cloud:** As the name suggest, this cloud is the combination of different type of clouds and are able to share and transfer data, applications and computations through their interfaces from one cloud to another. The hybrid cloud is an amalgamation of private and public clouds that support the requirement of public access with large scale resource sharing, storage along with ensuring some security features in clouds.

SERVICE MODELS

The service models of cloud computing emphasize on the types of services provided by the cloud to its users. It has incorporated the following three basic services:

- a) **Hybrid Software as a Service (SaaS):** The user can simply access software on internet instead of installing them in their system. SaaS is the global collections of all softwares to be used for different domains to develop various applications and to solve different problems.

- b) **Platform as a Service (PaaS):** It provides a development environment or platform as a service for the programmers so that they can develop and deploy their own applications and coding. It enables the programmer by providing them the liberty to develop application on the platform provided as a service by the provider.
- c) **Infrastructure as a Service (IaaS):** Infrastructure to be used in computing in the form of network, operating system, hardware and storage devices are provided as a service by this component of service model. The users demand for the necessary service in the form of infrastructure and the demand is catered by IaaS provider. IaaS users can access the services using a wide area network, such as the internet.

CHARACTERISTICS OF CLOUD COMPUTING

Characteristics that well define cloud's nature of computation are listed below:

- a) **On-demand self-service:** As the name suggests, this service ensures that there is no direct interactions occurring between there source consumer and resource provider at any time. When the consumer requests for a resource, there source availability is ensured automatically without any direct human interactions which make the process of resource consumption easy and convenient.
- b) **Broad network access:** This form of computation is build and delivered over the network, in essentially, the internet. The usage of different services and resources are enabled in the presence of good networking capabilities that ensures proper utilization of these services over various different computing platforms like PCs, mobile phones, work stations, PDA etc.
- c) **Resource pooling:** A cloud service provider follows mechanisms to pool together different resources dynamically to give an illusion that the resources available are not limited rather are vast to cater varying demands of different customers.
- d) **Rapid elasticity:** Here computing resources are immediately available to customer on their demand basis instead of holding on to the resource permanently. Users can use these resources and accordingly are able to scale up whenever they need, and release them once they finish. Virtualization enables the aggregation of different physical and virtual resources to serve different requests by scaling up resources. Hence it is the ability of cloud computing that enables provision of scalable computing resources.

- e) **Measured service:** There must be some mechanism that is able to calculate the cost of utilizing the resource. This metered service is enabled according to the pay as you use model that calculates the cost of resource utilization according to the consumption time. Hence the Service manager and resource provider are benefited by gaining profit from the amount incurred from the user.

ADVANTAGES OF CLOUD COMPUTING

There are numerous advantages to using cloud computing, which is why it has become increasingly popular among businesses and individuals. Some of the key advantages include:

- a) **Scalability:** Cloud computing allows users to scale resources up or down based on demand. This means you can quickly add or remove computing power, storage, or other resources as needed, without having to invest in and manage physical infrastructure.
- b) **Cost Efficiency:** Cloud computing typically operates on a pay-as-you-go model, where you only pay for the resources you use. This can be more cost-effective than purchasing and maintaining your own hardware, as you avoid up front capital expenses and only pay for what you need.
- c) **Flexibility and Accessibility:** Cloud computing provides flexibility for users to access applications and data from anywhere with an internet connection and on any device. This enables remote work, collaboration, and mobility, as users are not tied to specific physical locations or devices.
- d) **Reliability and Redundancy:** Cloud service providers often offer robust infrastructure and data redundancy measures to ensure high availability and reliability. This includes features such as data replication, backup, and failover mechanisms, which help mitigate the risk of data loss or service interruptions.
- e) **Scalable Storage:** Cloud storage solutions offer virtually unlimited storage capacity, allowing users to store large amounts of data without worrying about physical storage limitations. Additionally, cloud storage services often provide features such as data encryption, versioning, and access controls to enhance security and data management.

- f) **Automatic Updates and Maintenance:** Cloud service providers handle infrastructure maintenance, updates, and security patches, relieving users of the burden of managing these tasks themselves. This ensures that users have access to the latest features and security enhancements without having to allocate time and resources for maintenance activities.
- g) **Improved Collaboration:** Cloud computing facilitates collaboration among teams by providing centralized access to applications and data. Multiple users can work on the same documents or projects simultaneously, enhancing productivity and streamlining workflows.
- h) **Disaster Recovery and Business Continuity:** Cloud computing offers built-in disaster recovery and backup solutions, which can help organizations quickly recover from data loss or system failures. Cloud providers often have geographically distributed data centers and redundancy measures in place to ensure data durability and business continuity..

SCHEDULING

Scheduling in cloud computing refers to the efficient allocation and management of computing resources, such as virtual machines and tasks, to optimize performance and meet user requirements. It involves decisions on task placement, load balancing, and resource utilization to enhance overall system efficiency. Scheduling algorithms in the cloud aim to distribute workloads effectively, reduce latency, and maximize resource utilization, contributing to improved performance and responsiveness of cloud-based applications.

Scheduling can be done at job level or task level. Job scheduling is nothing but mapping and executing the whole job in a single resource whereas task scheduling is dividing the jobs into smaller tasks so that they can be executed in multiple resources simultaneously to reduce the total execution time.

There are several types of scheduling in cloud computing.

(i)Task Scheduling that involves scheduling individual tasks or jobs to available resources. This could include scheduling batch jobs, data processing tasks, or any other types of workloads.

(ii)Resource Scheduling which focuses on allocating physical or virtual resources, such as CPU, memory, storage, and network bandwidth, to tasks or services based on their requirements and the current state of the system.

(iii)Load Balancing which is a subset of resource scheduling, load balancing involves distributing incoming requests or workloads evenly across multiple servers or instances to ensure optimal resource utilization and prevent overloading any single server.

(iv)Workflow Scheduling which is used in scenarios where tasks have dependencies or form a workflow and this scheduling involves orchestrating the execution of these tasks in a way that meets dependencies and minimizes overall completion time.

Task scheduling can be done in static mode or dynamic mode. Static independent task scheduling in cloud computing refers to a scheduling approach where tasks are allocated to resources at the beginning of the execution without considering the dynamic changes in the system during runtime.

In this scenario, tasks are assumed to be independent of each other, and the scheduling decision is made based on the available information about the tasks and resources at the time of scheduling. This static scheduling method is suitable for applications with well-defined task characteristics and predictable resource requirements.

However, it may not adapt well to dynamic changes in workload or resource availability that can occur in a cloud environment. Dynamic and unpredictable workloads may benefit more from dynamic scheduling approaches that can adjust task assignments in real-time based on the changing conditions of the cloud infrastructure. Thus static scheduling is termed as compile time scheduling while dynamic scheduling is termed as run time scheduling.

The subdivided tasks can be dependent or independent. Dependent tasks should follow the priority while independent tasks need not follow the hierarchy and they can be executed in any order. The advantage of independent task scheduling is that it allows for efficient parallelization, where multiple tasks can be executed simultaneously on different resources. This approach can enhance system performance and reduce overall completion time.

However, it requires careful consideration of factors such as load balancing and resource availability to ensure optimal utilization of resources in a cloud environment. In the cloud computing environment virtual machines are considered as the resources and it should follow the policies to distribute the tasks among the virtual machines.

The set of policies and strategies that controls and govern the execution sequence of tasks to be performed by the system is termed as scheduling. Scheduling is an important aspect of any computing system.

In general, CPU scheduling deals with the execution of user-submitted jobs. First, all the user submitted jobs wait in ready queue for their turn of execution. The time spent in the ready queue by the job is known as waiting time. CPU scheduler selects jobs from the ready queue based on some criteria fulfilled by the job and assigns the same to the CPU for execution.

Scheduling itself has several parameters like execution time, resource utilization, fault tolerant, cost, load balancing and energy efficiency. It is intricate to schedule a set of submitted tasks set called meta-task, from diverse consumers on a computing environment to satisfy scheduling parameters. There are different types of algorithms existing for scheduling the core profit of scheduling algorithm is to get a high computing performance and the finest system throughput.

TASK SCHEDULING

Task scheduling is the action of mapping incoming user's tasks to available cloud resources (e.g. Virtual machines) within the suitable period of time for best resource utilization and good overall performance of cloud system based on the nature and characteristics of the given scheduling algorithm.

The overall performance of the scheduler depends on many parameters; reduction of makespan, best resource utilization, good distribution of workload among resources, waiting time, concurrent execution between small and large length tasks, etc. This also involves making decisions on how to distribute tasks among virtual machines or servers to optimize performance, minimize task completion time, and ensure effective resource utilization.

Task scheduling algorithms play a crucial role in determining the best allocation strategy based on factors like load balancing, priority, and resource availability in a dynamic and scalable cloud environment. Due to conflicting scheduling, it is difficult to offer an absolutely perfect scheduling algorithm. A good scheduler executes an appropriate agreement, or implements a mix of scheduling approaches according to diverse applications.

A problem can be solved in seconds, hours, or even years, depending on the algorithm used. The effectiveness of the algorithm is evaluated by the measure of time consumed to execute it.

Task scheduling is an essential and most important part in a cloud computing environment. The task scheduling mainly focuses to enhance the efficient utilization of resources and hence reduction in task completion time.

Task scheduling is used to allocate certain tasks to particular resources at a particular time instance. Many different techniques have been proposed to solve the problems of task scheduling. Task scheduling improves the efficient utilization of resource and yields less response time so that the execution of submitted tasks takes place within a possible minimum time.

The goal of the cloud computing service provider, is to use the efficient resources to obtain the maximum profit and this results the task scheduling a core and one of the challenging issues in the cloud environment. The scheduling strategy of a good task scheduler should get adopted based on the changing environments and the type of task scheduling used.

ASPECTS AND APPROACHES TO TASK SCHEDULING

Task scheduling in cloud computing involves allocating computing resources to execute tasks or jobs efficiently across a cloud infrastructure. The primary goal is to optimize resource utilization, minimize execution time, and meet various performance objectives.

Task scheduling algorithms play a crucial role in achieving these goals. Here are some common aspects and approaches to task scheduling in cloud computing:

1. **Task Characteristics Analysis:** Understanding the characteristics of tasks, such as their computational requirements, dependencies, deadlines, and priorities, is essential for effective scheduling. Different tasks may have varying resource demands and constraints, which influence scheduling decisions.
2. **Resource Availability Monitoring:** Task schedulers need to continuously monitor the availability of computing resources (e.g., virtual machines, containers) in the cloud environment. This involves tracking resource utilization, capacities, and constraints to ensure optimal allocation and scheduling of tasks.
3. **Scheduling Policies and Objectives:** Task schedulers employ various policies and objectives to guide scheduling decisions. These may include maximizing resource utilization, minimizing task completion time (e.g., makespan), meeting deadlines, balancing workload across servers, or minimizing energy consumption.
4. **Scheduling Algorithms:** There are different scheduling algorithms designed to address specific scheduling objectives and constraints:

- **First Come, First Served (FCFS):** Tasks are scheduled based on their arrival order. Simple but may not be optimal for minimizing task completion time.
 - **Shortest Job First (SJF):** Prioritizes tasks with the shortest estimated execution time. Helps minimize average response time but requires accurate task runtime estimates.
 - **Round Robin (RR):** Allocates fixed time slices to tasks in a cyclic manner. Ensures fairness but may lead to inefficient resource utilization for tasks with varying execution times.
 - **Priority-based Scheduling:** Assigns priorities to tasks based on their importance or urgency. Ensures critical tasks are executed promptly but may lead to starvation of low-priority tasks.
 - **Deadline-based Scheduling:** Ensures tasks meet their deadlines by scheduling them according to their deadlines. Requires accurate deadline estimation and may lead to resource under utilization.
 - **Genetic Algorithms, Ant Colony Optimization, etc.:** More complex optimization algorithms that leverage heuristics and metaheuristics to find near-optimal solutions for task scheduling problems.
5. **Dynamic Scheduling:** Cloud environments are dynamic, with varying workloads and resource availability. Dynamic scheduling algorithms adaptively adjust scheduling decisions based on real-time changes in workload, resource availability, and system conditions.
 6. **Fault Tolerance and Resilience:** Task schedulers may incorporate fault tolerance mechanisms to handle failures or resource unavailability during task execution. This involves rescheduling tasks to alternative resources or retrying failed tasks to ensure completion.
 7. **Load Balancing:** Task scheduling also involves load balancing to distribute tasks evenly across available resources, preventing resource bottlenecks and maximizing overall system performance

CHAPTER 2

LITERATURE SURVEY

S. Vaaheedha Kfatheen et al., 2021, An efficient scheduling algorithm rises the resource utilization thereby minimalizing the execution time and cost. This work proposes a Mean-Min Task Scheduling algorithm for Cloud environment based on the minimum execution time of the tasks submitted. The effectiveness and performance of proposed algorithm MMTS can be illustrated by conducting experiments with dissimilar number of tasks and virtual machines. Min-Min algorithm schedules the tasks based on minimum completion time which gives a fewer makespan that is less completion time. But it does not produce a balanced schedule. To overcome this limitation a new Task Scheduling algorithm MMTS is proposed which produces fewer makespan than the min-min algorithm as well as balances the load by efficiently utilizing the resources ^[1].

Ibrahim A. Thiyeb et al., 2020, This work suggested a new hybrid algorithm in cloud computing environment that based on two heuristic algorithms; Min-Min and Max-Min algorithms. To estimate this algorithm, the Cloudsim simulator has been used with dissimilar optimization parameters; makespan, load balancing, average of resource utilization, average of waiting time and concurrent execution among small length tasks and long size tasks. The outcomes show that the proposed algorithm is improved than the two algorithms Min-Min and Max-Min for those parameters. It provides greatest resource utilization, good makespan, efficient load balancing, better average waiting time, and best concurrent execution among long length tasks and small length tasks ^[2].

Hussin et al., 2016,proposed a hybrid algorithm based on Particle Swarm Optimization, Best-Fit and Tabu-Search algorithms called BFPSOTS. The initial population for PSO has been generated using Best-Fit instead of random generation. To improve the local search Tabu-Search has been used. To show the performance of the algorithm three different parameters execution time, resource utilization and cost have been considered. Through experiments the authors have shown that the proposed algorithm gives better results than the PSO algorithm^[3].

Kashish Ara Shakil et al., 2021, This work offers a latency-aware max-min algorithm (LAM) for allocation of resources in cloud infrastructures. The proposed algorithm was designed to address trials associated with resource allocation such as differences in user demands and on-demand access to infinite resources. It is capable of allocating resources in a cloud-based environment with the objective of improving infrastructure-level performance and maximization of profits with the optimal allocation of resources. A priority value is also connected with each user, which is calculated by analytic hierarchy process (AHP). The results confirm the superiority for LAM due to improved performance in comparison to other state-of-the-art algorithms with flexibility in resource allocation for fluctuating resource demand patterns. The proposed method can be simply incorporated into a real-world system as the experiments have been performed on a real world production workload i.e. Google cluster trace using system setup parallel to a public cloud such as Amazon EC2 ^[2].

T.Manoranjitham et al., 2020, In this work they proposed an effective task scheduling that uses three threshold values to require the resource to be allocated to a task at a given time. This method confirms that a capable resource is allocated to task such that the response time and makespan of the all task are minimized. The proposed method was simulated using CloudSim and the outcome shows an improved response time and makespan than the well-known Min-Min and Max-Min Method. Recently, a number of scheduling techniques have been proposed to minimize waiting time, execution time, makespan, and maximize resource utilization. Also, the techniques tend to promise QoS through task priority but fail to confirm the assignment of suitable resource to certify QoS delivery. In this work they proposed an effective scheduling technique that allots a capable resource to users tasks. The outcome of the simulation records less response time and makespan when compared with the well-known Min-Min andMax-Min method ^[5].

Deepika Saxena et al ., 2015, have developed an algorithm for task scheduling named Dynamic fair priority task scheduling algorithm for cloud computing. The newly developed algorithm groups the submitted tasks into two groups deadline based tasks and cost based tasks. Then these tasks are arranged in sorted order and mapped into three priority queues.^[6]

P Praveen Kumar et al., 2020, proposed a new task scheduling algorithm: "Optimized Flexi Max-Min Scheduling Algorithm" is proposed. The algorithm keeps a data structure which is modeled after a Binary Search Tree (BST) for estimations, improved searching, task allocation, and migration of tasks. CloudSim is used to model and pretend the cloud computing environment in order to attain simulated data. The outcome of this experiment shows that the proposed algorithm overtakes the traditional Max-Min task scheduling algorithms. Re-enacted their methodology exploiting CloudSim toolbox which vastly observed a regular decrease in holding up time prompting less makespan as progressively more cloudlets showed up to be planned. This is on the grounds that their methodology can select and allocate either cloudlet with greatest or least execution time to an asset. Execution examination, experimentation as well as benchmarking of the outcomes proves that their drew closer (MMax-Min) can create great quality arrangements, delivering great estimations of makespan and regulating load effectively when contrasted with the standard Max-Min and Round Robin calculations ^[7].

Pandaba Pradhan et al., 2020, The key thing they focused in this work is the technique of resource allocation that rules how the task should be scheduled and allocated to the resources for execution so as to decrease the average response time. The Min-Min algorithm is a static resource allocation method that assigns tasks with minimum execution time and thereafter tasks with maximum execution time. The restriction of existing algorithm is that tasks with longest execution time have to wait and there is stay in their execution. To eliminate these problems, the Efficient Min-Min approach is uttered and comparison among them is established. The experimented outcomes show that their Efficient Min-Min algorithm decreases the average response time by 45%. So, it is more efficient ^[8].

Amit Kaushal et al., 2019, Their proposed algorithm offer stronger hidden period from cloud supplier and reduce the holding up time for a specific shopper in cloud. Cloud computing could be a distributed chiefly based computer paradigm that is employed by users to make smart quality with less price. As task scheduling could be a test in cloud computing, totally dissimilar algorithms are quick and applied to induce higher results regarding utilization of system resources, latent period and satisfaction of user demands. During this study, they tend to highlight a number of the task programming algorithms working in cloud supported by examples

definitely genetic formula and Auction primarily based task scheduling algorithm. They tend to study the performance of recent Hybrid task scheduling inside the setting of CloudSim and compared among the performance of the Max–min scheduling formula and Genetic algorithm according to average holding up time. The outcomes show, once victimization new Hybrid algorithm to scheduling range of Cloudlets over number of VMs, the average waiting time develops less than Max-Min, using the matching numbers of cloudlets and therefore the CC environments^[9].

Hongyan Cui et al .,2017, have proposed a task scheduling algorithm based on Genetic Algorithm and Ant Colony Optimization. The newly proposed algorithm is named as Genetic Algorithm-Chaos Ant Colony Optimization (GA-CACO) which gives better results in terms of convergence speed and performance.^[10]

Mohammed A.S.Mosleh et al., 2016, have designed a Cost based Task Scheduling (ACTS) which gives data access to the virtual machines within the user specified deadline by minimizing the cost. In this algorithm low priority tasks are allocated minimum cost path and high priority tasks are allocated fast access path.^[11]

M.Zhu et al., 2012, have designed a delay-constrained optimization algorithm to increase the utilization of resources. They have developed a two-step workflow scheduling technique to reduce the cloud overhead within the specified time bound.^[12]

Tarun Biswas et al., 2019, In this work, they have proposed two resource aware multi- criteria scheduling, where scheduling is done by considering real-time resources such as, memory capacity, clock frequency, and residual energy. The proposed algorithms are designed using four conflicting objectives, viz., and minimization of energy-consumption, makespan, load-balancing, and maximization of resource utilization. The proposed work is analyzed and confirmed by wide simulations with synthetic as well as benchmark data sets. Through the simulation outcomes, it is observed that the proposed work has substantial improvements over Min–Min, MCT, Genetic algorithm (GA) and priority based performance improved algorithm (PPIA). A statistical hypothesis test Analysis of Variance (ANOVA) and post hoc test are performed to reveal the efficiency of the proposed work.

The proposed work has considerable development in terms of makespan, energy-consumption, load- balancing and resource- utilization. Their experimental results are certified using synthetic as well as standard level datasets over the Min–Min, MCT, GA, and PPIA ^[13].

Belal Ali Al-Maytami et al., 2019, They present a novel scheduling algorithm using Directed Acyclic Graph (DAG) based on the Prediction of Tasks Computation Time algorithm (PTCT) to guesstimate the preeminent scheduling algorithm for prominent cloud data. In addition, the proposed algorithm offers a significant improvement with respect to the makespan and decreases the computation and complexity via employing Principle Components Analysis (PCA) and decreasing the Expected Time to Compute (ETC) matrix. Simulation outcomes confirm the higher performance of the algorithm for heterogeneous systems in terms of efficiency, speedup and schedule length ratio, when compared to the state-of-the-art Min-Min, Max-Min, QoS-Guide and MiM-MaM scheduling algorithms. The primary aim of this research is to advance the performance of task scheduling, while decreasing computational costs. A main objective is to predict the perfect algorithm for incoming/available data as and when needed. In order to attain this, they perform a systematic analysis of heuristic techniques for resource utilization by means of Principal Components Analysis (PCA) in the cloud environment. Moreover, they analyze the requirements as well as consequences of utilizing Quality of Service (QoS) with the proposed Prediction of Tasks Computation Time algorithm (PTCT) ^[14].

D. I. George Amalarethinam et al., 2019, This work proposes a new REMM algorithm. This strategy chooses the allocation of tasks in Meta task set based on speed of resources in cloud environment. Min-Min strategy is suitable for small range distributed environment. If tasks with less run time rise in number, than task with high run time in a meta-task, the Min-Min algorithm cannot assign the tasks proficiently, and the makespan is moderately large. LBMM algorithm overcomes restrictions of Min-Min at some extent but it produces the same makespan as Min-Min for some problem sets. New Rescheduling Enhanced Min-Min Meta task scheduling algorithm is proposed to eradicate the annoyances of LBMM. The investigational outcomes of the Rescheduling Enhanced Min-Min algorithm offer evidence for its improved performance than the Min-Min and LBMM. REMM is concerned about the resources and run time of the task ^[15].

Sabreena Hamid et al., 2019, In this work, it introduced an Optimized task Scheduling Algorithm which will adjust the advantages of numerous algorithms and uses them according to the condition while following the suitable procedure. The outcomes are presented and compared with Existing Min-Max algorithm. Max-Min and Min-Min are related in slightly scale cloud systems. At the point when the volume of vast tasks is greater than smaller tasks or the other way around, both computation tasks may not be appropriately scheduled and makespan usually becomes large. Improved Max-Min and Enhanced Max-Min try to complete load-adjusting among resources by scheduling enormous tasks before smaller ones yet again and end up in makespan expansion. The optimized task scheduling algorithm gives an intelligent computational technique in which the system itself adjusts the greatest task scheduling scheme from the current Max-Min and Enhanced Max-Min calculations specified by the condition. In outcomes, it have seen that Periodic tasks are scheduling by numerous scheduling algorithms. By simulation outcomes it is shown that the proposed algorithms always performs better than that of Basic performance parameter consideration ^[16].

CHAPTER 3

ANALYSIS AND DRAWBACKS IN EXISTING SYSTEM

In the existing system, a Min-Min algorithm was proposed. The min-min task scheduling algorithm is a static task scheduling algorithm used for load balancing in cloud computing. It first finds the minimum execution time of all tasks and then chooses the task with the least execution time among all the tasks. The algorithm proceeds by assigning the task to the resource that produces the minimum completion time. This process is repeated until all tasks are scheduled.

The min-min has two phases namely, Resource Discovery and Resource Selection. In the resource discovery phase, minimum ETC for each task is found. Then the task with overall minimum ETC is selected and mapped to the resource that produces it. This second phase is termed as resource selection.

The min-min algorithm helps in deciding which task to assign to which virtual machine, ensuring that the load is balanced and the resources are used efficiently. However, it considers the shortest jobs first, which may lead to a load imbalance as resources may not be utilized efficiently.

The min-min task scheduling algorithm is a simple and fast algorithm that produces a better makespan but may not utilize the resources efficiently due to its focus on shortest jobs first.

The min-min task scheduling algorithm in cloud computing has some drawbacks. One of the main drawbacks is that it considers the shortest jobs first, which may lead to a load imbalance as resources may not be utilized efficiently. This can result in some resources being idle while others are heavily loaded.

CHAPTER 4

STATEMENT OF THE PROBLEM

The performance of cloud computing will be improved if an effective task scheduling algorithm in cloud environment is used for mapping the tasks to the available resources. An effective scheduling algorithm in cloud environment increases the resource utilization thus minimalizing the execution time and cost.

Every user needs his task to be accomplished with the minimum completion time, so a good scheduler is that performs the scheduling and allocation of users concurrently & fairly without any interruption of some tasks or users.

Task scheduling is the action of mapping received user's tasks to available cloud resources (e.g. Virtual Machines) within the correct period of time for best resource utilization as well as good complete performance of cloud system based on the nature and characteristics of the given scheduling algorithm.

The whole performance of the scheduler depends on numerous parameters like decrease of makespan, top resource utilization, good dispersion of workload among resources, wait time, concurrent execution among small and large length tasks, cost etc.

In the existing Min-Min algorithm, scheduling process is done by considering the minimum execution time. It first finds the minimum execution time of all tasks and then chooses the task with the least execution time among all the tasks.

The algorithm proceeds by assigning the task to the resource that produces the minimum completion time. This process is repeated until all tasks are scheduled. This algorithm considers the shortest jobs first, which may lead to a load imbalance as resources may not be utilized efficiently.

This can result in some resources being idle while others are heavily loaded. In order to overcome these, An Efficient Task Scheduling Algorithm in Cloud Environment is proposed for better outcomes.

CHAPTER 5

SCOPE OF THE RESEARCH

There are numerous issues and challenges in cloud computing like security, cost, accessibility, completion time of user tasks, performance monitoring, flexibility, availability, wants a constant and speedy Internet connection, consistent as well as robust service abstractions, interoperability and portability, resource allocation and migration, task scheduling, scale and QoS management, and effective load balancing.

Most of researchers consider task scheduling, resource allocation and load balancing are the key concerns in the cloud computing and distribution systems, they play a vital role to develop overall system performance. Task scheduling of arriving user's tasks among the cloud resources is the main concern in the cloud computing to achieve improved performance of cloud system.

Like in existing methods, there is no allocation of shortest task first in “An Efficient Task Scheduling Algorithm in Cloud Environment”, so there is no load imbalance and high makespan. Task scheduling in proposed algorithm is set based on segregation of tasks into two groups. Subsequently the task is scheduled alternatively from each group.

Based on a constant threshold value all the tasks are segregated into two groups. The first group contains all the tasks which has the minimum execution time less than or equal to the threshold value and the second group contains all the tasks which has the minimum execution time more than the threshold value.

In proposed algorithm, the minimum execution time of task will be allotted to the resources alternatively from each group. Improved resource utilization and reduced makespan can be attained by using proposed algorithm.

CHAPTER 6

OBJECTIVES OF THE RESEARCH

Task scheduling is one of the key problems in Cloud computing. Here we have proposed an algorithm which schedules the task based on a threshold value. Based on the threshold value the tasks are segregated into two groups and based on the group the tasks are allocated to the resources.

The proposed algorithm focuses on maximum resource consumption as well as minimum makespan in cloud computing, than the existing algorithms. The following are the important objectives of the proposed algorithm.

- To meet the necessities of cloud providers and users concurrently with limited to the computational time.
- To allot tasks depending upon the threshold value, which will utilizes all the resources in a best way.
- To decrease the makespan, load unbalancing, time complexity, wait time, and running time and rise utilization of resources.

CHAPTER 7

PROPOSED METHODOLOGY

AN EFFICIENT TASK SCHEDULING ALGORITHM IN CLOUD ENVIRONMENT

In the proposed methodology, users submit the tasks with its time. Then the ETC (Expected Time to Complete) matrix is calculated. The completion time CT_{ij} of each task is calculated using the following equation:

$$CT_{ij} = ET_{ij} + R_i$$

Where ET_{ij} is the expected execution time of i th task on j th resource and R_i is the ready time of the resource. The ready time is also called as the availability time of resource j and it is calculated from the completion time of previously assigned tasks..

The min-min has two phases namely, Resource Discovery and Resource Selection. In the resource discovery phase, minimum ETC for each task is found. Then the task with overall minimum ETC is selected and mapped to the resource that produces it. This second phase is termed as resource selection phase.

In this proposed algorithm, ETC has been taken as input for the overall process. From the ETC the Threshold Value (TV) is calculated. The median of minimum execution of each task in any of the given resources is taken as the TV. The below steps are used to calculate the Threshold Value.

- Take the minimum execution time of all the tasks
- Arrange the minimum execution time of all the tasks in ascending order.
- Count the number of tasks(n).
- If the number of task is odd, then the formula $[(n + 1)/2]$ th term is used to find the median. If the number of task is even, then the formula $([(n/2)]\text{th term} + ((n/2) + 1)\text{th term})/2$ is used to find the median.

After calculating TV, the given ETC Matrix is divided into two ETCs. One is ETC1 and another one is ETC2. Tasks having Execution time lesser than or equal to TV are added to ETC1 and Tasks having Execution Time greater than TV are added to ETC2.

After segregating the ETC into ETC1 and ETC2 the allocation of tasks in the resources is done based on the tasks present in both ETCs. The minimum execution time of tasks are allocated alternatively from both ETCs. The allocation tasks initially starts by allocating the first task in ETC1 which was followed by the first task in ETC2. Then the second task in ETC1 is allocated which was followed by the second task in ETC2.

This process continues until all the tasks are allocated in the resources.

ALGORITHM

initialize tables that contains task and resources, etc1, etc2

initialize variables totalTasks, totalResources, median

//Get total number of tasks and resources from the user

input totalTasks, totalResources

//Get the execution time of each task in each resource

for i = 1 to totalTasks

for j=1 to totalResources

input tij (execution time of task i in resource j)

append tij to taskTable[i][j]

//Calculate the minimum execution time of each task

minExecutionTimes = []

for i = 1 to totalTasks

minExecutionTime = infinity

for j = 1 to totalResources

if tij < minExecutionTime


```
minExecutionTime = tij
append minExecutionTime to minExecutionTimes
```

//Calculate the median of minimum execution times sort minExecutionTimes in ascending order

```
Median = minExecutionTimes[(totalTasks+1)/2]
```

//Create etc1 and etc2 based on the median value

```
for i = 1 to totalTasks
for j=1 to totalResources
if minExecutionTimes[i] <= median
append tij to etc1[i][j]
else
append tij to etc2[i][j]
```

//Allocate the minimum execution time of tasks alternatively from etc1 and etc2

```
allocationTable = [ ]
for i=1 to totalTasks
row = [ ]
for j=1 to totalResources
if table1[i][j] is not empty
append table1[i][j] to row
remove table1[i][j] from table1
else
append table2[i][j] to row
remove table2[i][j] from table2
append row to allocationTable
```

WORK FLOW OF THE PROPOSED ALGORITHM

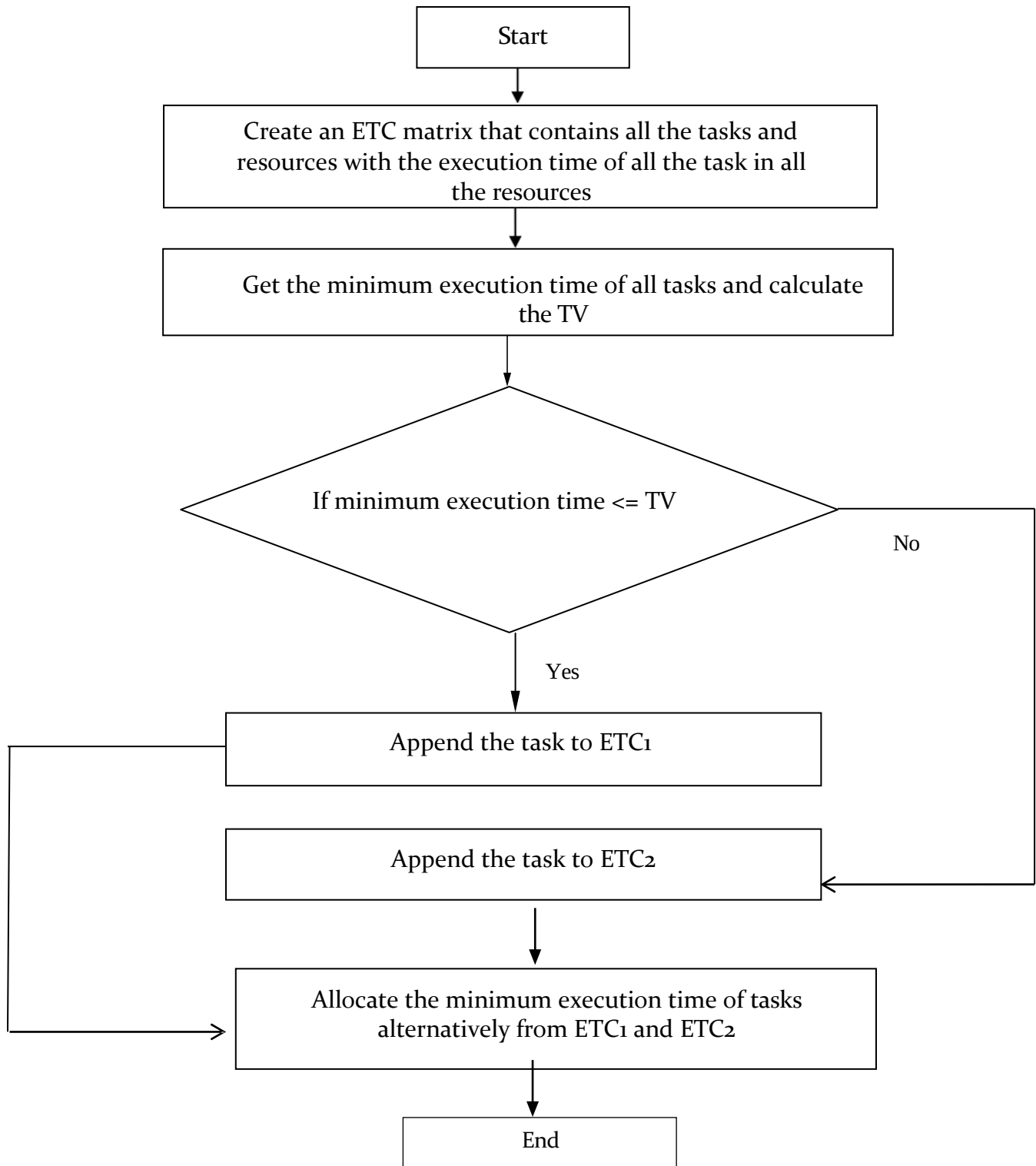


Figure1.Work flow of the Proposed Algorithm

PERFORMANCE PARAMETERS

The performance of result is assessed according to the below parameters:

- a) **Makespan (MS):** Makespan is defined as the time necessary for completing all the tasks. In other words, it can be defined as the maximum time required by the system running over the data center. If the makespan is less, the load balancing also better in result. One of the main characteristics of good task scheduling algorithm is minimizing makespan. Makespan is calculated by using equation (1)

$$MS = \text{Max} (C_Time_{ij}) \quad \dots(1)$$

Where C_Time_{ij} is the completion time of tasks T_i on resource R_j

- b) **Resource Utilization (RU):** Resource Utilization is a concept that describes the degree of use of resources in the system. If load balancing is maximum that means resource utilization is maximized. The rate of resource utilization is calculate by using equation (2)

$$RU_j = \sum T_i \text{ where } T_i \text{ will be executed on } r_j (t_{\text{end}(i)} - t_{\text{start}(i)}) \quad \dots(2)$$

where, $t_{\text{end}(i)}$ — finishing time of task T_i on resource r_j

$t_{\text{start}(i)}$ — starttime of task T_i on resource r_j

r_j — resource

- c) **Load Balancing (LB):** Load Balancing is the best dispersal of work loads among all cloud resources in effective manner. Load Balancing should be high, for effectual and best task scheduling algorithm. Load in virtual machine is calculated by using equation (3)

$$\text{Load (VM)} = \frac{\sum_{i=0}^n T_{Li}}{C(\text{VM})} \quad \dots(3)$$

CHAPTER 8

EXPERIMENTAL RESULT

Three different set of ETC Matrices are taken to prove the accuracy of proposed algorithm.

Table 1 is a Sample ETC Matrix1 which contains 9 Tasks and 3 Resources. Task T1 needs 4.1 unit of time to finish its work on resource R1, it needs 5 unit of time to finish its work on resource R2 and it needs 7.2 unit of time to finish its work on resource R3. Likewise Task T2 needs 2.6 unit of time to finish its work on resource R1, it needs 3 unit of time to finish its work on resource R2 and it needs 3.1 unit of time to finish its work on resource R3.

The execution time of other tasks on every resource is given in table. The given ETC matrix is scheduled using the proposed algorithm.

Table 1.Sample ETC Matrix

Task	R1	R2	R3
T1	4.1	5	7.2
T2	2.6	3	3.1
T3	6.7	8	8.2
T4	9.2	15	10.5
T5	5.9	7	6.8
T6	3.5	4	6.2
T7	7.8	8.2	10
T8	4.5	6	5.8
T9	8.9	9	13

As in algorithm Threshold Value is calculated first. To do so the Minimum execution time of tasks on every resource is taken and median of them is calculated.

$$TV = [(9+1)/2]\text{th term} = 5^{\text{th}} \text{ term}$$

Table 2 contains the minimum execution time of all tasks in ascending order and the median value is the 5th term which is 5.9.

Table 2. Minimum Execution time of all tasks

T2	2.6
T6	3.5
T1	4.1
T8	4.5
T5	5.9
T3	6.7
T7	7.8
T9	8.9
T4	9.2

The Minimum execution time of every task on every resource is compared with the Threshold Value. If any of the task having the lesser or equal minimum execution time on any resource is added into ETC matrix1 and the tasks having higher execution time is added into ETC matrix2.

The Table 3 shows the ETC matrix1 and Table 4 shows the ETC matrix2.

Table 3.ETC matrix1

<u>ETC matrix1</u>			
Task	R1	R2	R3
T2	2.6	3	3.1
T6	3.5	4	6.2
T1	4.1	5	7.2
T8	4.5	6	5.8
T5	5.9	7	6.8

Table 4.ETC matrix2

<u>ETC matrix2</u>			
Task	R1	R2	R3
T3	6.7	8	8.2
T7	7.8	8.2	10
T9	8.9	9	13
T4	9.2	15	10.5

The minimum execution time of tasks are allocated alternatively from both ETCs. The allocation tasks initially starts by allocating the first task in ETC1 which was followed by the first task in ETC2. Then the second task in ETC1 is allocated which was followed by the second task in ETC2. This process continues until all the tasks are allocated in the resources.

Table 5 shows the allocation of the tasks in the resources using the proposed algorithm

Table 5.Scheduling using the proposed algorithm

Tasks	R1	R2	R3
T2	2.6	-	-
T3	-	8	-
T6	3.5	-	-
T7	-	-	10
T1	4.1	-	-
T9	-	9	-
T8	4.5	-	-
T4	-	-	10.5
T5	5.9	-	-
Completion time	<u>20.6</u>	17	20.5

Table 6.Scheduling using the Min-Min Algorithm

Tasks	R1	R2	R3
T2	2.6	-	-
T6	-	4	-
T8	-	-	5.8
T1	4.1	-	-
T5	-	7	-
T3	6.7	-	-
T7	-	-	10
T9	-	9	-
T4	9.2	-	-
Completion time	<u>22.6</u>	20	15.8

The scheduling using Min-Min algorithm is shown in Table 6 and the scheduling based on proposed algorithm is shown in Table 5. Min-Min produces a makespan of **22.6** while the proposed algorithm produces a makespan of **20.6**

From the results it is observed that proposed algorithm produces less makespan than Min-Min algorithm. The schedule is also shown as chart in Figure 2 and Figure 3. The proposed algorithm effectively utilizes the resources.

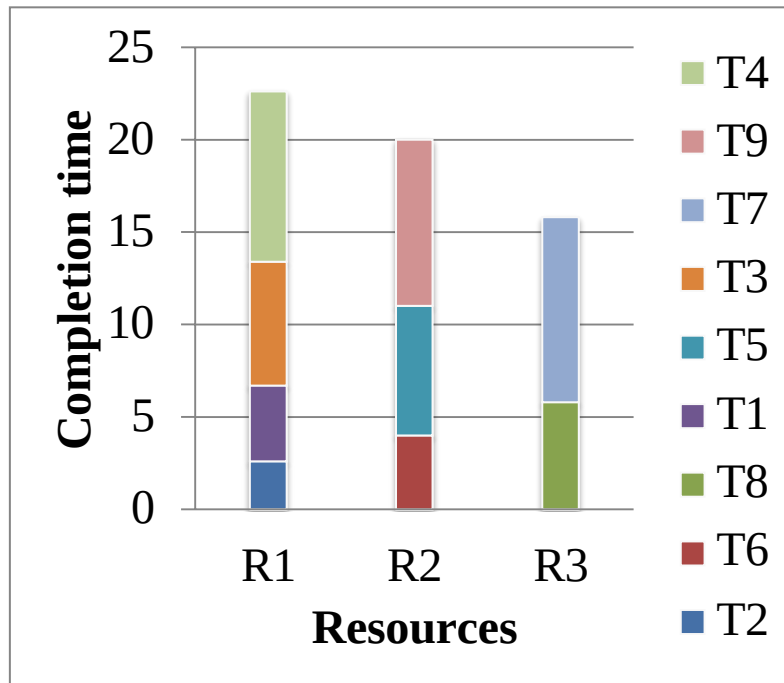


Figure 2. Chart of Min-Min algorithm

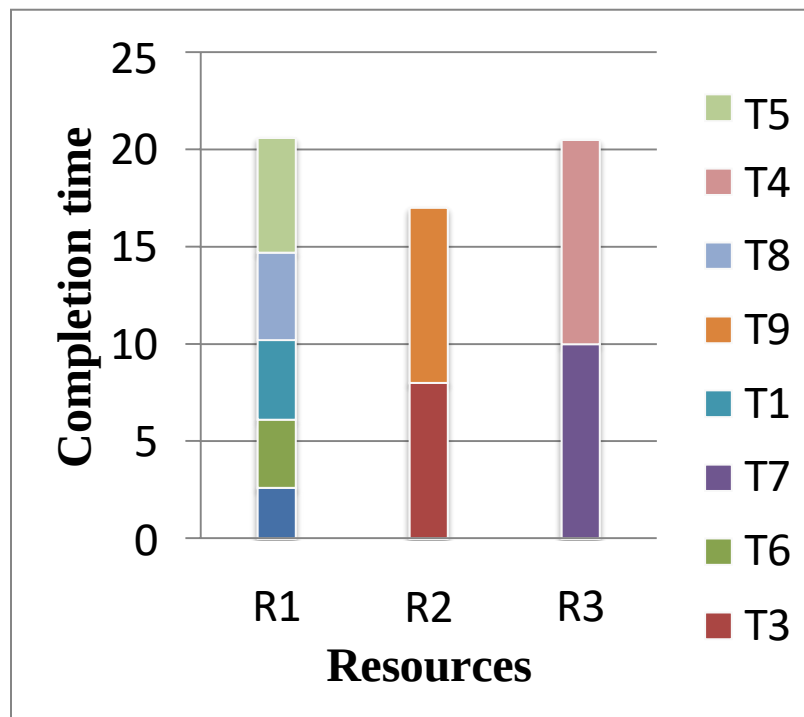


Figure 3. Chart of the proposed algorithm

CHAPTER 9

CONCLUSION AND FUTURE WORK

One of the main concern in cloud computing is task scheduling of incoming user's tasks among the cloud resources to achieve better performance of cloud system.

Cloud computing is a form of distributed technology used by millions of users. The performance of cloud computing can be increased in terms of time and cost by effectively utilizing the resources.

Min-Min algorithm schedules the tasks based on minimum completion time which gives a less makespan. But it does not produce a balanced schedule.

To overcome this limitation a new Task Scheduling algorithm is proposed which produces less makespan than the min-min algorithm as well as effectively utilizes the resources.

The task scheduling using the proposed algorithm result achieves maximum resource utilization, efficient load balancing and minimum makespan only the resource utilization and makespan has been concentrated in this paper.

But in future both the computation and communication costs can also be calculated and it can be shown that the proposed algorithm will also reduce the cost.

REFERENCES

- [1] S. Vaaheedha Kfatheen, “Mean-Min task scheduling algorithm for cloud environment”, *Malaya Journal of Matematik*, Vol. 9, No. 1, 2021.
- [2] Ibrahim A. Thiyeb and Sharaf A. Alhomdy, “HAMM: A Hybrid Algorithm of Min-Min and Max-Min Task Scheduling Algorithms in Cloud Computing”, *International Journal of Recent Technology and Engineering (IJRTE)*, Vol. 9, Issue. 4, November 2020.
- [3] Hussin M. Alkhashai, Fatma A. Omara, “An Enhanced Task Scheduling Algorithm on Cloud Computing Environment”, *International Journal of Grid and Distributed Computing*, 9(7)(2016), 91-100, ISSN: 2005-4262 IJGDC.
- [4] Kashish Ara Shakil, Mansaf Alam and Samiya Khan, “A latency-aware max-min algorithm for resource allocation in cloud”, *International Journal of Electrical and Computer Engineering (IJECE)*, Vol. 11, No. 1, February 2021.
- [5] T. Manoranjitham and Dupati Srikar, “Efficient Task Scheduling for Quality of Service in Cloud Computing Network”, *International Journal of Recent Technology and Engineering (IJRTE)*, Vol. 8, Issue. 5, January 2020.
- [6] Deepika Saxena, Dr. R.K. Chauhan, “A Review On Dynamic Fair Priority Task Scheduling Algorithm In Cloud Computing”, *International Journal of Science, Environment and Technology*, ISSN 2278-3687 (O), 3(3)(2014),997-1003.
- [7] P Praveen Kumar and V Kanchana Devi, “Efficient load balancing by optimized flexi max-min algorithm”, *National Science, Engineering and Technology Conference (NCSET)*, 2020.
- [8] Pandaba Pradhan, Prafulla Ku. Behera and B. N. B. Ray, “Efficient Min-Min Algorithm for Resource Allocation in Cloud Computing”, *International Journal for Research in Engineering Application & Management (IJREAM)*, Vol. 06, Issue. 08, November 2020.
- [9] Amit Kaushaland Sarvpal Singh, “Performance Optimization by Task Scheduling in Cloud Computing”, *International Journal of Engineering and Advanced Technology (IJEAT)*, Vol. 9 Issue. 1S, October 2019.

- [10] D. I. George Amalarethinam and S. Kavitha, “Rescheduling Enhanced Min-Min (REMM) Algorithm for Meta-task Scheduling in Cloud Computing”, Springer, pp. 895–902, January 2019.

APPENDIX

A. SOURCECODE

main.py

```
import numpy as np

def get_input():

    n = int(input("Enter the number of tasks:"))

    m = int(input("Enter the number of resources:"))

    task_names = [f"t{i+1}" for i in range(n)]

    resource_names = [f"r{i+1}" for i in range(m)]

    execution_times = np.zeros((n, m))

    for i in range(n):

        for j in range(m):

            execution_times[i,j] = float(input(f"Enter the execution time of task
                                                {task_names[i]} in resource {resource_names[j]}:"))

    return task_names, resource_names, execution_times

def display_table(task_names, resource_names, execution_times):

    print("\nTask".ljust(10), end=" ")

    for resource in resource_names:
```

```

    print(resource.ljust(10),end=" ")

    print()

for i in range(len(task_names)):

    print(task_names[i].ljust(10),end=" ")

for j in range(len(resource_names)):

    print(f"{execution_times[i,j]:.2f}".ljust(10),end=" ")

    print()

def get_min_execution_times(execution_times):

    min_execution_times = np.min(execution_times, axis=1)

    min_execution_times_indices = np.argmin(execution_times, axis=1)

    return min_execution_times, min_execution_times_indices


def allocate_tasks(task_names, resource_names, execution_times,
    min_execution_times, min_execution_times_indices):

    allocated_tasks = []

    allocated_resources = []

while len(allocated_tasks) < len(task_names):

    for i in range(len(task_names)):

        if i not in allocated_tasks:

```



```

min_execution_time = min_execution_times[i]

min_execution_time_index = min_execution_times_indices[i]

if min_execution_time <= median:

    allocated_tasks.append(i)

    allocated_resources.append(min_execution_time_index)

    min_execution_times[i] = np.inf min_execution_times_indices[i] = -1

    return allocated_tasks, allocated_resources

def display_allocated_tasks(task_names, resource_names, allocated_tasks,
allocated_resources):

    print("Task".ljust(10), end=" ")

    for resource in resource_names:

        print(resource.ljust(10), end=" ")

        print()

    for i in range(len(allocated_tasks)):

        print(task_names[allocated_tasks[i]].ljust(10),end=" ")

    for j in range(len(resource_names)):

        if j == allocated_resources[i]:

            print(f"{execution_times[allocated_tasks[i],j]:.2f}".ljust(10),end=" ")

```


else:

```
print(" ".ljust(10),end=" ")
```

```
print()
```

```
if __name__=="__main__":
```

```
    task_names, resource_names, execution_times = get_input()
```

```
    display_table(task_names, resource_names, execution_times)
```

```
    min_execution_times, min_execution_times_indices =
```

```
        get_min_execution_times(execution_times)
```

```
    print("\nMinimum execution times:")
```

```
    print("Task".ljust(10), end=" ")
```

```
    for resource in resource_names:
```

```
        print(resource.ljust(10), end=" ")
```

```
        print()
```

```
    for i in range(len(task_names)):
```

```
        print(task_names[i].ljust(10),end=" ")
```

```
    for j in range(len(resource_names)):
```

```
        print(f"{min_execution_times[i]:.2f}".ljust(10),end=" ")
```

```
        print()
```

```
    median = np.median(min_execution_times)
```

```

    print(f"\nMedian: {median}")

    table1_tasks, table1_resources = [], []

    table2_tasks, table2_resources = [], []

    for i in range(len(task_names)):

        if min_execution_times[i] <= median:

            table1_tasks.append(task_names[i])

            table1_resources.append(min_execution_times_indices[i])

        else:

            table2_tasks.append(task_names[i])

            table2_resources.append(min_execution_times_indices[i])

    print("\nTable1:")

    display_table(table1_tasks, resource_names, execution_times)

    print("\nTable 2:")

    display_table(table2_tasks, resource_names, execution_times)

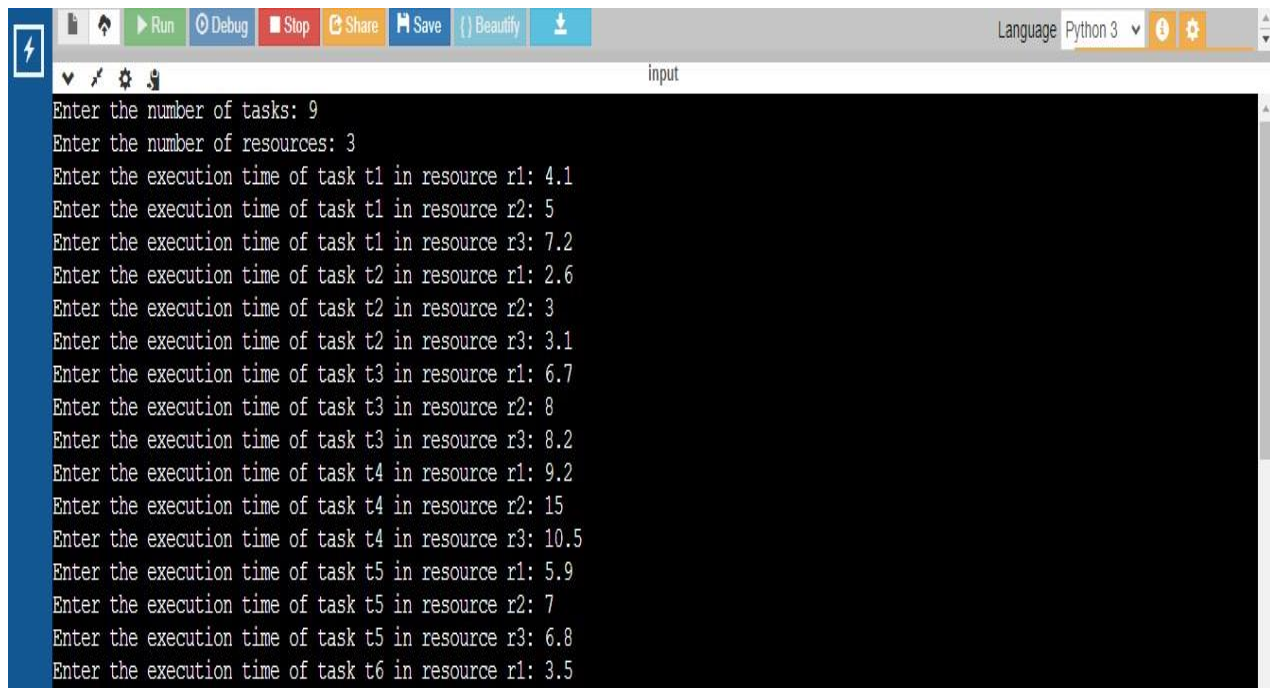
    allocated_tasks, allocated_resources = allocate_tasks(task_names, resource_names,
        execution_times, min_execution_times, min_execution_times_indices)

    print("\nAllocatedtasks:")

    display_allocated_tasks(task_names, resource_names, allocated_tasks,
        allocated_resources)

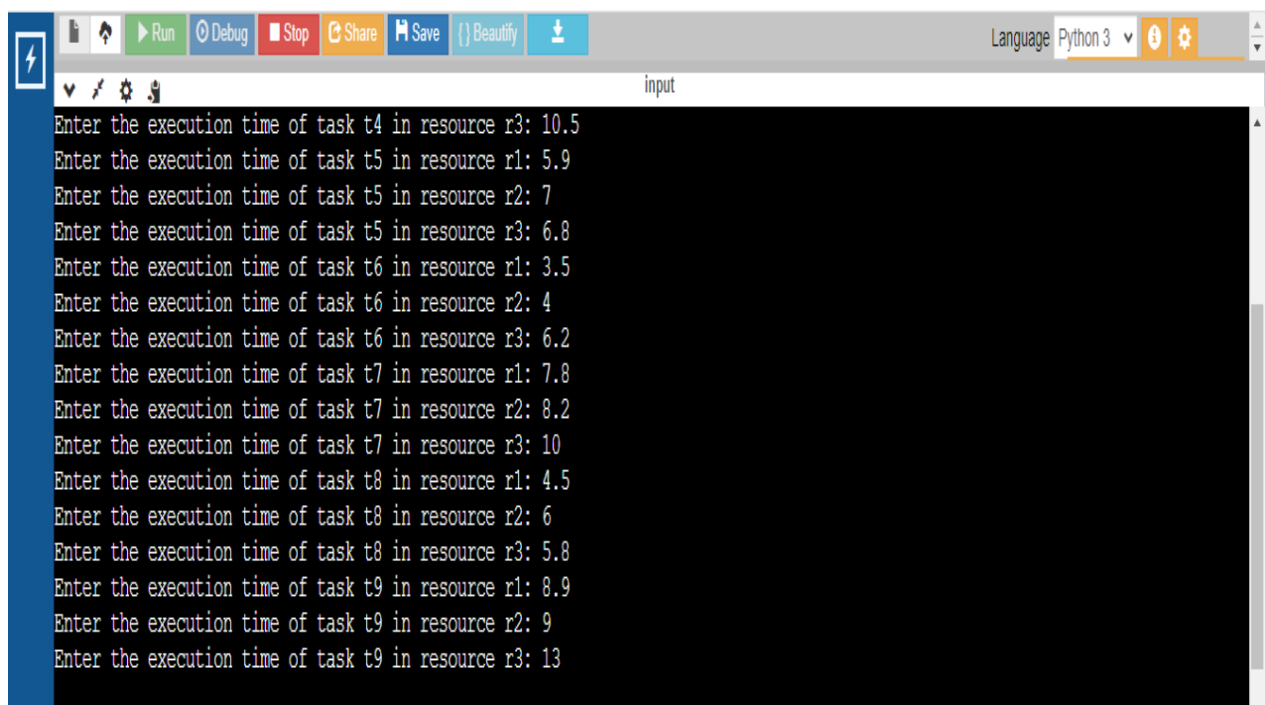
```

B. SCREEN INTERFACES:



A screenshot of a Python IDE terminal window. The window has a title bar with a lightning bolt icon on the left and a toolbar with buttons for Run, Debug, Stop, Share, Save, and Beautify. The language is set to Python 3. The terminal output shows a series of prompts for entering execution times for 9 tasks across 3 resources. The prompts are as follows:

```
input
Enter the number of tasks: 9
Enter the number of resources: 3
Enter the execution time of task t1 in resource r1: 4.1
Enter the execution time of task t1 in resource r2: 5
Enter the execution time of task t1 in resource r3: 7.2
Enter the execution time of task t2 in resource r1: 2.6
Enter the execution time of task t2 in resource r2: 3
Enter the execution time of task t2 in resource r3: 3.1
Enter the execution time of task t3 in resource r1: 6.7
Enter the execution time of task t3 in resource r2: 8
Enter the execution time of task t3 in resource r3: 8.2
Enter the execution time of task t4 in resource r1: 9.2
Enter the execution time of task t4 in resource r2: 15
Enter the execution time of task t4 in resource r3: 10.5
Enter the execution time of task t5 in resource r1: 5.9
Enter the execution time of task t5 in resource r2: 7
Enter the execution time of task t5 in resource r3: 6.8
Enter the execution time of task t6 in resource r1: 3.5
```



A screenshot of a Python IDE terminal window, similar to the one above. The terminal output shows prompts for entering execution times for tasks 4 through 9 across 3 resources. The prompts are as follows:

```
input
Enter the execution time of task t4 in resource r3: 10.5
Enter the execution time of task t5 in resource r1: 5.9
Enter the execution time of task t5 in resource r2: 7
Enter the execution time of task t5 in resource r3: 6.8
Enter the execution time of task t6 in resource r1: 3.5
Enter the execution time of task t6 in resource r2: 4
Enter the execution time of task t6 in resource r3: 6.2
Enter the execution time of task t7 in resource r1: 7.8
Enter the execution time of task t7 in resource r2: 8.2
Enter the execution time of task t7 in resource r3: 10
Enter the execution time of task t8 in resource r1: 4.5
Enter the execution time of task t8 in resource r2: 6
Enter the execution time of task t8 in resource r3: 5.8
Enter the execution time of task t9 in resource r1: 8.9
Enter the execution time of task t9 in resource r2: 9
Enter the execution time of task t9 in resource r3: 13
```

```
Enter the execution time of task t9 in resource r2: 9
Enter the execution time of task t9 in resource r3: 13

Task    r1    r2    r3
t1      4.10  5.00  7.20
t2      2.60  3.00  3.10
t3      6.70  8.00  8.20
t4      9.20  15.00 10.50
t5      5.90  7.00  6.80
t6      3.50  4.00  6.20
t7      7.80  8.20  10.00
t8      4.50  6.00  5.80
t9      8.90  9.00  13.00
```

```
Minimum Execution Time Table:
Minimum Execution Time
t2      2.6
t6      3.5
t1      4.1
t8      4.5
t5      5.9
t3      6.7
t7      7.8
t9      8.9
t4      9.2
```

```
Median: 5.9

Table 1:

Task    r1    r2    r3
t1      4.10  5.00  7.20
t2      2.60  3.00  3.10
t5      6.70  8.00  8.20
t6      9.20  15.00 10.50
t8      5.90  7.00  6.80

Table 2:

Task    r1    r2    r3
t3      4.10  5.00  7.20
t4      2.60  3.00  3.10
t7      6.70  8.00  8.20
t9      9.20  15.00 10.50
```

```
17 table1[0][1] = min(r1[0] + r2[0] + r3[0])

Task r1 r2 r3
t2 2.6 0.0 0.0
t3 0.0 8 0.0
t6 3.5 0.0 0.0
t7 0.0 0.0 10
t1 4.1 0.0 0.0
t9 0.0 9 0.0
t8 4.5 0.0 0.0
t4 0.0 0.0 10.5
t5 5.9 0.0 0.0

...Program finished with exit code 0
Press ENTER to exit console.
```

Activate Windows
Go to Settings to activate Windows.